arXiv:2410.13276v4 [cs.CL] 17 Feb 2025

SeerAttention: Learning Intrinsic Sparse Attention in Your LLMs

Yizhao Gao [*1] **Zhichen Zeng** [*2] **Dayou Du** [3] **Shijie Cao** [4] **Peiyuan Zhou** [5] **Jiaxing Qi** [5] **Junjie Lai** [5]
Hayden Kwok-Hay So [1] **Ting Cao** [4] **Fan Yang** [4] **Mao Yang** [4]

Abstract

Attention is the cornerstone of modern Large Language Models (LLMs). Yet its quadratic complexity hinders efficiency and scalability, especially for long-context processing. A promising approach is to leverage sparsity in attention. However, existing sparsity-based solutions predominantly rely on *predefined patterns or heuristics* at the attention head level, struggling to adapt dynamically to different contexts efficiently.

We propose SeerAttention, a simple yet effective attention mechanism that directly learns the block-level attention sparsity from the LLM itself. Inspired by the gating mechanism in Mixture of Experts (MoE), SeerAttention augments the conventional attention with a **learnable gate** that **selectively activates important blocks** within the attention map. Specifically, the gate first pools the query (Q) and key (K) tensors along the sequence dimension and processes them through learnable linear layers. The resulting matrices are then multiplied together to produce the gating scores, which are used to predict block-level attention sparsity. Combined with our block-sparse FlashAttention kernel, SeerAttention can achieve significant speedup on GPUs. When applied to pre-trained LLMs, SeerAttention only requires training the gate parameters in a lightweight self-distillation manner, allowing rapid convergence. Our evaluation results demonstrate that SeerAttention achieves better model accuracy and lower latency for long-context pre-filling compared to prior methods. Code is available at: https://github.com/microsoft/SeerAttention

[*]Equal contribution [1]The University of Hong Kong [2]University of Washington [3]Hong Kong University of Science and Technology (Guangzhou) [4]Microsoft Research [5]NVIDIA. Correspondence to: Shijie Cao <shijiecao@microsoft.com>.

1. Introduction

Attention is a fundamental mechanism in transformer-based LLMs (Vaswani, 2017). Despite its effectiveness, the quadratic complexity of attention demands substantial computation and memory resources, limiting the scalability and efficiency of LLMs, especially for long-context windows. This challenge has become an active research topic in the community. One potential solution is to replace the quadratic attention with cheaper architectures like linear attention or recurrent networks (Katharopoulos et al., 2020; Gu & Dao, 2023; Peng et al., 2023) with subquadratic complexity. While these approaches are more efficient, the majority of state-of-the-art large language models (LLMs) continue to use full attention to achieve better performance.

A promising approach with increasing interests is to leverage sparsity in attention. Sparsity commonly exists in attention maps, and it becomes more prominent in longer contexts. In certain LLM attention heads, the sparsity ratio can reach 95% or even 99%, posing great opportunities for efficiency improvements. However, prior studies often rely on predefined sparsity patterns or heuristics to approximate the attention mechanism (Jiang et al., 2024; Fu et al., 2024; Lee et al., 2024; Zhu et al., 2024; Han et al., 2023; Xiao et al., 2024). The sparsity observed in attention maps varies significantly across different models, input contexts and attention heads, making predefined patterns or heuristics insufficient.

In this paper, we introduce SeerAttention, a simple yet effective attention mechanism that directly learns the intrinsic attention sparsity from the LLM itself, without relying on predefined sparsity patterns. To achieve this, SeerAttention augments conventional attention with a learnable gate that selectively activates a small subset of important blocks in the attention map, drawing inspiration from the gating mechanism in MoE (Shazeer et al., 2017). The gating process in SeerAttention consists of three key steps. First, the query ($\mathbf{Q}$) and key ($\mathbf{K}$) matrices are pooled along the sequence length to reduce the substantial gating cost while preserving essential information. Second, the pooled $\mathbf{Q}$ and $\mathbf{K}$ representations are processed through linear layers to enable learnability. Finally, the transformed representations are multiplied to compute gating scores, which adaptively

identify the most important blocks in the attention map. By skipping unimportant blocks, the resulting block-sparse attention significantly reduces both I/O overhead and computational cost.

To train the gating mechanism in SeerAttention, we adopt a lightweight self-distillation approach. Specifically, the pooled attention map from standard attention serves as the teacher, guiding block-level sparsity learning in the SeerAttention gate, which acts as the student. Importantly, for pre-trained LLMs, SeerAttention only requires learning the gating parameters, while all other model parameters remain fixed. This leads to fast training process as only the newly added gate weights requires to compute gradient. For instance, when applied to a Llama-3.1-8B-Instruct model, SeerAttention uses only 0.5B tokens for gate distillation, which is approximately 40 A100 GPU hours for training.

Our results demonstrate that SeerAttention surpasses state-of-the-art sparse attention methods like Minference (Jiang et al., 2024), MoA (Fu et al., 2024) and DuoAttention (Xiao et al., 2024) in terms of long context model accuracy and prefilling latency. SeerAttention achieves highly linear speedup over dense configurations, delivering a 7.3× speedup with 90% sparsity on sequences of 128k. Notably, in contrast to previous methods that require careful calibration of sparse configuration for different heads, SeerAttention offers strong capabilities of adaptation to different heads and contexts. Remarkably, on top of block-sparse pattern, SeerAttention exhibits the ability to learn more diverse patterns, including A-shape and Vertical-Slash, further demonstrating its versatility and performance.

Our contributions can be summarized as follows:

- We propose SeerAttention, an innovative learnable attention gating mechanism to enhance efficiency for long-context LLMs.
- We have developed a self-distillation training scheme to efficiently train the AttnGate, enabling it to learn the intrinsic sparsity of a pre-trained model.
- Experiments show that SeerAttention outperforms previous approaches, offering adaptability to various context lengths and sparsity ratios.

2. Background and Related Works

Powerful but Complex Attention in Transformer. The advent of attention mechanisms, particularly within the Transformer architecture (Vaswani, 2017), marked a significant advancement in natural language processing. Attention enables improved handling of long-range dependencies and a better understanding of context by attending each token to every other token in the sequence, resulting in a quadratic memory and time complexity $O(n^2)$, where n is the sequence length. This presents a significant challenge as the community moves towards LLMs that can process increasingly longer contexts. Many studies explore alternative attention mechanisms to mitigate this complexity. The Reformer architecture (Kitaev et al., 2020) reduces the complexity to $O(n \log n)$ and the linear attention mechanism (Katharopoulos et al., 2020; Yang et al., 2023) further decreases complexity to $O(n)$. Recently, there has been a trend of revisiting recurrent neural networks, leading to the proposal of new architectural frameworks such as RWKV (Peng et al., 2023), RetNet (Sun et al., 2023), and Mamba (Gu & Dao, 2023). Despite their promise of efficiency, these methods struggle to match the performance of full attention mechanisms, particularly with larger models and longer contexts.

Intrinsic but Dynamic Sparsity in Attention. Attention mechanisms inherently exhibit sparsity, which arises from the attention map $\mathbf{A}$ generated by $\mathbf{Q}$ and $\mathbf{K}$: $\mathbf{A} = \mathrm{softmax}(\mathbf{Q}\mathbf{K}^{\mathbf{T}}/\sqrt{d})$. The softmax function often produces a multitude of negligible scores that can be treated as zeros without impacting model accuracy (Zaheer et al., 2020; Liu et al., 2021; Wang et al., 2021; Child et al., 2019; Liu et al., 2023). Attention sparsity becomes more pronounced with longer contexts, presenting opportunities to optimize inference speed. Unfortunately, this sparsity is dynamic, varying across different context inputs and attention heads, each displaying distinct sparsity locations and ratios. Prior research has attempted to approximate attention sparsity using predefined patterns and heuristics (Fu et al., 2024; Jiang et al., 2024) for different attention heads. Yet, these methods lack generality and often rely on handcrafted features, struggling to fully capture the sparsity behavior of attention mechanisms. The dynamic and input-dependent nature of attention sparsity echoes the principles of Mixture of Experts (MoE) models (Shazeer et al., 2017; Fedus et al., 2022) suggesting that sparsity should ideally be learned directly from data within the model itself. This approach would allow models to adaptively harness sparsity, improving efficiency while maintaining accuracy.

Long-Context LLM Optimizations. The ability to process long contexts is crucial for large language models (LLMs) as it enables them to retain and utilize more extensive information. However, it comes with substantial computational and memory costs. Various research efforts have explored different strategies to optimize long-context processing. One major direction is improving prefill efficiency, where techniques such as prompt compression (Jiang et al., 2023; Mu et al., 2024; Chuang et al., 2024) and sparse attention (Jiang et al., 2024; Fu et al., 2024; Acharya et al., 2024a; Zhang et al., 2024b). Another approach focuses on optimizing the decoding phase by introducing sparse loading

mechanisms (Yang et al., 2024; Chen et al., 2024). Additionally, several methods aim to compress the KV cache, including KV cache sharing (Ainslie et al., 2023; Brandon et al., 2024), KV eviction policies (Zhang et al., 2023; Li et al., 2024; Ge et al., 2023), and KV quantization (Liu et al., 2024; Hooper et al., 2024; Dong et al., 2024; Zhang et al., 2024a).

3. SeerAttention

SeerAttention adopts a fully learning-based approach to adaptively identify attention sparsity in LLMs and leverages the learned sparsity for efficient inference. To ensure efficiency on modern hardware like GPUs, we focus on learning block sparsity, which can seamlessly integrate with the tiling computation scheme of FlashAttention (Dao et al., 2022; Dao, 2023). Figure 1 illustrates the overall diagram of SeerAttention, which augments conventional attention with a learnable gating module, termed *Attention Gate* (AttnGate). The AttnGate modules contain learnable parameters (linear layers) and are distilled to mimic the 2D-Maxpooled results of the attention maps. At inference time, the AttnGate can predict the block-level sparsity for the subsequent attention computation with a block-sparse FlashAttention kernel, which significantly enhances performance by reducing I/O and computation overhead.

3.1. Attention Gate Design

The AttnGate module is designed to learn block-wise information with minimal overhead. It takes the original matrices $\mathbf{Q}$ and $\mathbf{K}$ as inputs and downsamples them using pooling operations along the sequence dimension. As shown in Figure 1, for a given attention head, the sizes of the pooled $\mathbf{Q}$ and $\mathbf{K}$ become $[seq/B, d]$, where B is the kernel and stride size of the pooling operation (non-overlapped blocks). The downsampled $\mathbf{Q}$ and $\mathbf{K}$ are then processed through a linear layer and multiplied together, similar to the standard attention operation. This results in a matrix of size $[seq/B, seq/B]$, where each element corresponds to one block in the original full attention map. With a typical block size of 64, the output of the AttnGate module is only $\frac{1}{4096}$ the size of the original attention map, making it super efficient to compute. To its simplest form, the AttnGate computation can be expressed as:

$$score = \text{softmax}\left(\frac{(W_q\, P_q(Q)) \cdot (W_k\, P_k(K))^T}{\sqrt{d}}\right) \quad (1)$$

where P_q and P_k represents the pooling operations for $\mathbf{Q}$ and $\mathbf{K}$, and d is the hidden size of the tensors similar to attention computation.

Pooling Method Selection. As pooling operations downsample the tensors and might lead to information loss, in SeerAttention, we allow different pooling methods to be composed for the $\mathbf{Q}$ and $\mathbf{K}$ tensors to better perverse their characteristics. We use the combinations of average, max, and min pooling. When appling more than one pooling methods on either $\mathbf{Q}$ or $\mathbf{K}$, the resulting pooled tensors will first be concatenated in the hidden dimension before being fed into the linear layer. Figure 2 shows the test perplexity on PG19 (Rae et al., 2019) datasets with the best 15 pooling combinations using the Llama-3.1-8B model. Results shows that using avgpooling on $\mathbf{Q}$ and a combination of max, min, avg pooling on $\mathbf{K}$ achieves best perplexity on across different sparsity ratios. This observation might be related to the phenomenon in LLM quantization that $\mathbf{K}$ tensors tend to have more outliers. Thus, with the aid of Max and Min pooling can better extract the features in $\mathbf{K}$ after pooling.

Block-level RoPE in AttnGate. Modern LLMs typically employ RoPE (Su et al., 2024) to encode positional information. If the AttnGate relies solely on the original RoPE in the model, i.e., feeding the AttnGate with $\mathbf{Q}$ and $\mathbf{K}$ after RoPE, the relative positional encoding properties will be lost because of the pooling operation (shown in Figure 3a). This compromises the AttnGate's ability to extrapolate to longer context lengths during AttnGate distillation. To address this issue, we propose a different design by feeding the AttnGate with $\mathbf{Q}$ and $\mathbf{K}$ before RoPE encoding, and add an additional Block-level RoPE in AttnGate (shown in Figure 3b). To represent the block-level information, the new RoPE in AttnGate uses a reduced $\theta' = \theta/B$, where θ is the original RoPE theta.

Figure 4 presents the test perplexity results with and without the block-level RoPE design in AttnGate. The results indicate that without this block-level RoPE design, AttnGate fails to perform adequately on evaluation data longer than 8k when trained with 8k length data. Similarly, when trained with 64k length data, it does not perform well on 128k length data. However, with the additional block-level RoPE, AttnGate can extrapolate to different context lengths, significantly enhancing the model performance and training efficiency.

3.2. AttnGate Distillation in SeerAttention

While the introduced SeerAttention architecture is straightforward, training presents challenges. Jointly training the gate and model from scratch, as in MoE, is costly and difficult. Fortunately, unlike MoE, where gating network must learn expert selection from scratch, the AttnGate in SeerAttention has a ground truth from standard attention for distillation.

Obtaining the Ground Truth. We use the 2D-MaxPooled attention map from full attention as ground truth to distill AttnGate, as illustrated in Figure 1. Semantically,

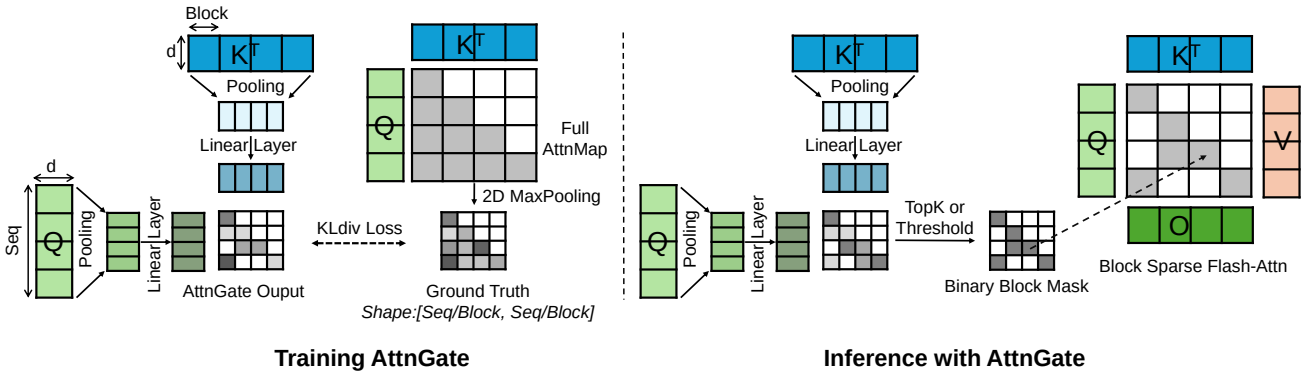


Figure 1. **Overall of SeerAttention.** The AttnGate in SeerAttention first pools the Q and K tensors in sequence dimension and passes through learnable linear layers. The matrix multiplied results are then trained to mimic the 2D maxpooled results of the original pre-trained LLM in a self-distillation manner. During inference, the gating score can be used to predict the block-level attention sparsity through TopK or Thresholding. Noted that details gate design like pooling composition, RoPE and softmax are omitted in this diagram for simplicity.

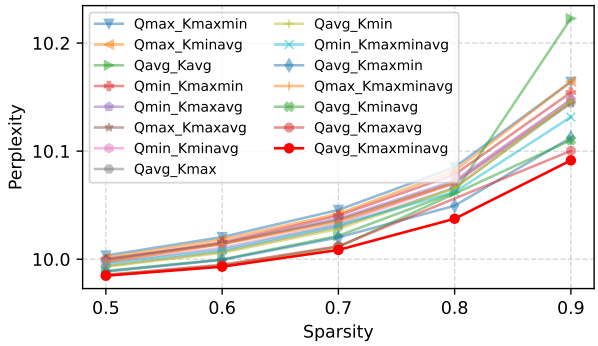


Figure 2. **Test Perplexing of Different Pooling Method Combinations on PG19.** The best configuration observed is using AvgPooling on Q and a combination of Max, Min, AvgPooling on K.

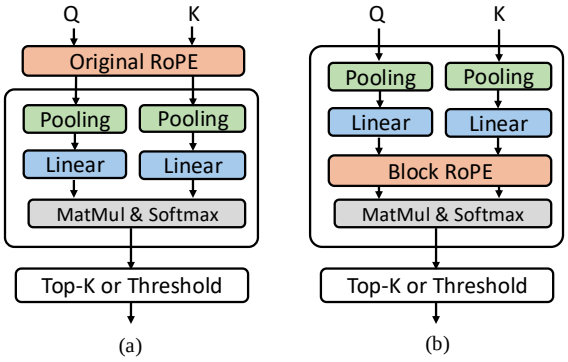


Figure 3. **Two Different RoPE Design in AttnGate.** (a) Directly taking the RoPE encoded Q and K as AttnGate input. (b) Using the Q and K before RoPE as AttnGate input and perform an block-level RoPE after linear layers. Results shows that (b) performs better and does not overfit to training data length. For simplicity, pooling composition is omitted in this diagram.

it means that only when all the attention score in a block is small, the 2D-MaxPooled results will be small. This is aligned with the block-sparse definition. However, obtaining the max-pooled attention map for training is non-trivial especially in long-context scenarios. Modern LLMs rely on FlashAttention, which fuses operations between layers and doesn't explicitly compute the attention map. The naïve manual implementation is impractical due to quadratic memory consumption. To address this challenge, we customize an efficient kernel that directly outputs the MaxPooled attention map ground truth by modifying FlashAttention kernel but largely reuses its original computation flow. The detailed design are explained in Appendix A.1.

Loss Function. The Kullback-Leibler divergence loss (Joyce, 2011) is use to distill the AttnGate. Since AttnGate uses softmax in output similar to full attention computation, the row summation of gating score will always be 1. KL-divergence loss allows the training process to focus on mimicking the attention distribution instead of absolute magnitude like Mean-square-error loss. The overall distillation process can be expressed as:

$$\begin{aligned} gt &= \mathrm{MaxPool2D}\Big(\mathrm{softmax}\big(\tfrac{QK^T}{\sqrt{d}}\big)\Big), \\ score &= \mathrm{AttnGate}(Q, K), \\ loss &= D_{KL}\big(gt \parallel score\big). \end{aligned} \tag{2}$$

3.3. Inference with SeerAttention

After self-distillation training process, SeerAttention can utilizes the trained AttnGate to generate a gating score for each block within the full attention mechanism. These scores are then used to select the final activated sparse blocks. By integrating with our Block-sparse FlashAttention kernel, SeerAttention can achieve significant speedup for long-context prefilling while maintaining high accuracy.

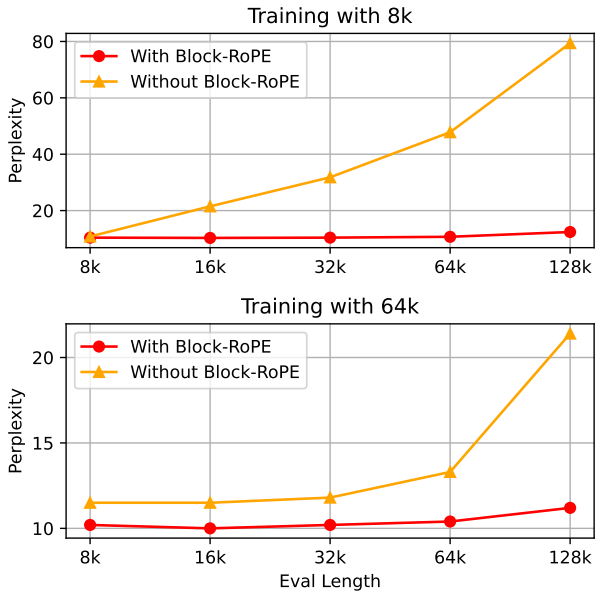


Figure 4. **Perplexity Comparison Between two RoPE design in AttnGate on PG19 dataset.** The block-level RoPE in AttnGate allows it to effectively learn the block-level positional information, resulting better test performance for different context length. The baseline design (Figure 3a) fails to deliver reasonable results with data longer than training length.

Generating Binary Block Mask. SeerAttention provides the flexibility to convert the floating-point gating scores into a final binary block mask using either the TopK or Thresholding methods. If using the TopK method, the k blocks with the highest scores in each row are selected.

$$b_{ij} = \begin{cases} 1 & \text{if } j \in \text{TopK}(score_i, k).\text{index}, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

Alternatively, users can apply a gating threshold to activate blocks with scores exceeding the specified threshold.

$$b = score > threshold \tag{4}$$

Notably, once AttnGate is trained, users can adjust the TopK ratio or threshold at test time to achieve various trade-offs.

Block Sparse Flash-Attn Kernel. In designing the Block Sparse Flash-Attention kernel, the block size of AttnGate is aligned with the tiling size used in Flash-Attention, typically 64 or 128. By doing so, we can create a customized block-sparse Flash-Attention kernel that leverages the binary block mask generated by AttnGate to selectively skip the I/O and computation for unactivated blocks. This approach is highly efficient on modern GPUs, as it optimizes the processing of sparse data at the block level rather than dealing with fine-grained element-wise level, leading to significant performance gains.

4. Experiments

In this section, we evaluate both the accuracy and efficiency of SeerAttention. In our current experiments, block-size B for the AttnGate and sparse kernel is fixed at 64 and AttnGate solely applies in the prefill stage.

Models, Tasks and Baselines. We apply SeerAttention to the pre-trained models Llama-3.1-8B-Instruct (Dubey et al., 2024) in the following experiments. We evaluate the AttnGate performance using perplexity test on PG19 (Rae et al., 2019), two long context benchmarks: LongBench (Bai et al., 2023) and RULER (Hsieh et al., 2024), and 4 short-context task from Open LLM Leaderboard (Tunstall et al., 2023): HellaSwag (Zellers et al., 2019), MMLU (Hendrycks et al., 2020), ARC-challenge (Clark et al., 2018), GSM8K (Cobbe et al., 2021). For long-context benchmark like RULER and LongBench, we follow similar practice in Acharya et al. that only applies sparsity in context rather than question in SeerAttention. We compare SeerAttention with three state-of-the-art sparse attention methods, MoA (Fu et al., 2024), MInference (Jiang et al., 2024), and DuoAttention (Xiao et al., 2024). MoA uses an offline search scheme to apply static sparse patterns across different attention heads. In our experiment, we adopt their "KV Sparsity" in 0.5 which means "Attention Sparsity" in 0.35. MInference dynamically generates sparse indices using heuristic methods for each head based on predefined sparse patterns. We used their official configuration for Llama-3.1-8B-Instruct model, where all attention heads choose the "Vertical-Slash" sparsity pattern. DuoAttention differentiates some attention heads as streaming heads (Xiao et al., 2023) while keep the rest as dense heads. In the following experiment, we adopted their official setup for Llama-3.1-8B-Instruct model with 50% head as streaming heads. All the evaluation were run on a single A100.

Distillation Training Setup. We use the RedPajama (Computer, 2023) dataset for AttnGate distillation, which are chunked into 64k with BOS and EOS tokens. Our training employs a learning rate of 1e-3 with cosine decay. We set the global batch size to 16 and conduct training for only 500 steps, leveraging DeepSpeed (Rasley et al., 2020) stage 2 optimization on A100 GPUs. As only AttnGate parameters are learned and updated, the distillation process can be completed within 40 A100 hours. To prevent the quadratic memory explosion that occurs when saving the intermediate attention map for ground truth generation, we customized a FlashAttention kernel. This kernel directly outputs the 2D max-pooled ground truth on top of the original attention computation. Further details about this kernel can be found in Appendix A.1.

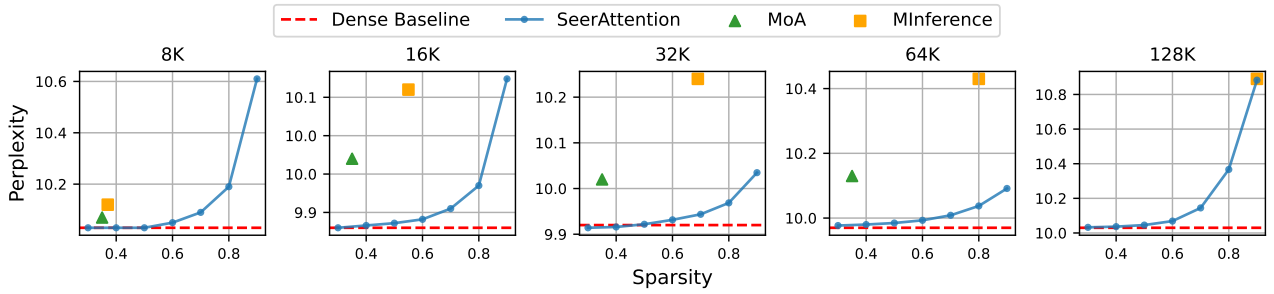


Figure 5. **Perplexity results on PG19 test split**. Each figure shows the Perplexity results of MoA, MInference, and SeerAttention using LLama-3.1-8B-Instruct model under different evaluation context size (MoA got OOM issue on 128k evaluation). SeerAttention allows users to adjust different sparsity ratio at test time and demonstrates better tradeoff.

Table 1. LongBench Results on Llama-3.1-8B-Instruct Model.

	0-4k	4-8k	8k+	Avg. Acc.	Avg. Sparsity
Full Attention	55.32	53.98	52.9	54.07	0.0
MInference	55.23	53.78	52.18	53.73	0.31
MoA	50.74	49.84	51.89	50.82	0.35
DuoAttention	53.77	52.17	51.27	52.40	0.5*
SeerAttention	**55.43**	**54.49**	**52.69**	**54.20**	**0.50**

* 50% streaming heads, the real sparsity <50%

4.1. Accuracy of Evaluation

Perplexity Results. Figure 5 presents the perplexity results on the PG19 test split across various evaluation lengths. All documents exceeding 128k tokens from PG19 test split are selected, and the input sequences are truncated to evaluation context length before evaluation. We compare the performance of SeerAttention with MoA and MInference. Notably, with a single trained AttnGate, users have the flexibility to adjust the TopK or threshold values during testing, allowing them to achieve different trade-offs between accuracy and efficiency. The figure illustrates the results for different TopK sparsity levels of SeerAttention, where we uniformly apply the same TopK ratio to all attention heads. The results indicate that SeerAttention generally offers a better trade-off compared to MoA and MInference, achieving lower perplexity at similar sparsity ratios. Note that the MoA's result for 128k is missing due to an Out-of-memory (OOM) issue on a single A100 GPU.

LongBench Evaluation. LongBench is a long-context understanding benchmark. We compare with those of MoA, MInference, and DuoAttention using the Llama-3.1-8B-Instruct model. DuoAttention uses 50% of the heads as streaming heads and 50% as dense heads. For streaming heads, the attention only occurs in the attention sink and recent tokens. As a result, it is not less than 50% sparsity overall. In this benchmark test, SeerAttention employs a threshold of 2e-3 for all AttnGates. With the same threshold, different attention gates can exhibit varying sparsity ratios, and longer context data tends to be sparser. This approach allows for a more adaptive allocation of sparsity. As demonstrated in Table 1, SeerAttention consistently outperforms other methods across various test lengths. Notably, in the 0-4k and 4-8k tests, our score surpasses even the dense baseline. This may be attributed to AttnGate filtering out noisy attention in certain cases. Furthermore, SeerAttention achieves the highest average score and the highest average sparsity across all tests.

RULER Evaluation. RULER is a long-context LLM evaluation benchmark consisting of 13 challenging sub-tasks. It generates tests with data sizes ranging from 4k to 128k. In this experiment, SeerAttention employs a threshold of 5e-4, which allows it to automatically adapt sparsity from approximately 10% for 4k data to around 85% for 128k data. Due to out-of-memory (OOM) issues in some tests, MoA was excluded from this benchmark. Table 2 provides detailed accuracy results across different evaluation lengths. It is evident that SeerAttention achieves the best accuracy in most tests (8k-64k). For the 128k test, DuoAttention has less than 50% sparsity, whereas SeerAttention maintains an sparsity higher than 80%, which accounts for the slightly lower performance. SeerAttention also attains the highest average accuracy compared to other models (only 0.41% lower than the dense baseline) while delivering the highest average end-to-end speedup ($1.41\times$) in prefilling time.

Short Context Test. For short context input, attention contributes a smaller proportion in the total runtime. Consequently, sparse attention does not significantly enhance latency performance. Nevertheless, we evaluate SeerAttention accuracy performance under a very high threshold 3e-2 to achieve high sparsity. The results, as shown in Table 3, indicate that SeerAttention exhibits negligible accuracy loss. For instance, with an average sequence length of 872 in the GSM-8K task, SeerAttention achieves only 0.1% degradation in accuracy with 52% averaged sparsity.

Table 2. RULER Benchmark Results on Llama-3.1-8B-Instruct Model.

Methods	4k	8k	16k	32k	64k	128k	Average Accuracy	Average Speedup
Full Attention	95.53	92.37	92.01	87.63	84.39	76.26	88.01	1.00
MInference	95.53	92.64	91.37	85.71	83.24	67.02	85.92	0.83
DuoAttention	**95.64**	92.08	90.71	84.75	83.24	**75.32**	86.96	1.09
SeerAttention	95.53	**92.71**	**92.02**	**88.49**	**83.48**	73.37	**87.60**	**1.41**

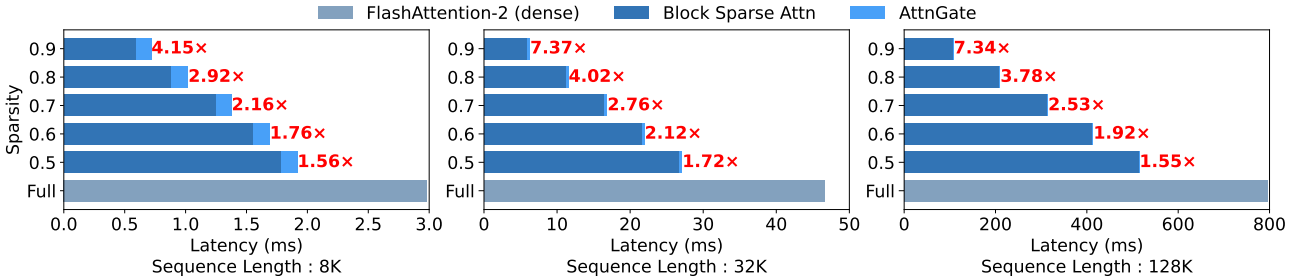


Figure 6. **SeerAttention Speedup over FlashAttention-2 at the Kernel Level.** The latency overhead from AttnGates is minimal. Our block-sparse attention kernel achieves highly linear speedup over dense configurations, delivering a 7.3× speedup with 90% sparsity on sequences of 128k. The AttnGate overhead amost diminishes in 128k context length.

Table 3. Short Context Tests on Llama-3.1-8B-Instruct Model

	MMLU	HellaS.	ARC-c	GSM-8K
Full Attention	68.1	80.1	60.7	75.7
SeerAttention	67.9	79.8	60.2	75.6
Avg Sparsity	3.4	50.4	26	52.1
Avg Seqlens	118	840	395	872

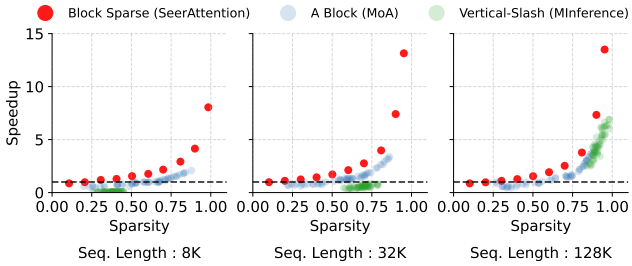


Figure 7. Kernel-level Speedup Comparison Between Different Works. SeerAttention translates sparsity to speedup more effectively.

4.2. Efficiency Evaluation

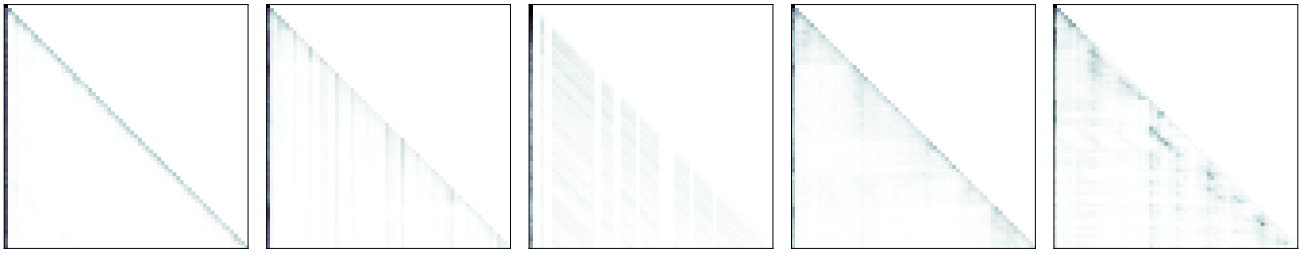
We evaluate the efficiency of SeerAttention using our implementation of CUDA kernels. We evaluate the kernel-level as well as end-to-end speedup using a Llama-3.1-8B-Instruct on a single A100 GPU. Results are compared to FlashAttention-2 (dense baseline), MoA, MInference and DuoAttention.

4.2.1. Kernel Evaluation

Negligible Overhead incurred by AttnGate. Figure 6 shows the kernel-level latency breakdown of SeerAttention. It demonstrates that the overhead introduced by the AttnGate during inference is minimal. For instance, at a context length of 32K and a sparsity of 0.5, the AttnGate contributes only 1% to the total latency of an attention layer. In the cases of 128K sequence length, the relative overhead almost diminishes.

Block-sparse FlashAttention Kernel Speedup. Figure 6 also shows that our kernel exibits linear speedup over various sparsity levels. At a sequence length of 128K with 90% sparsity, SeerAttention achieves a speedup of 7.3× compared with FlashAttention-2 (full attention) on a single A100 GPU. This demonstrates the effectiveness of the block-level sparsity employed by SeerAttention, which is highly efficient on GPUs and translates into high speedup.

Kernel-level Comparison with Related Works. We compare the kernel-level speedup of SeerAttention with MoA and MInference. MInference uses offline calibration to identify a pre-defined sparse pattern for each layer. For Llama-3.1-8B-Instruct model, MInference consistently uses "Vertical-slash" pattern across all layers. During runtime, MInference will dynamically generate non-zero indices based on their approximation algorithm. On the other hand, MoA uses "A-shape" blocks as their sparse pattern and calibrate the shape parameters offline under given sparsity constraint. DuoAttention is omitted in kernel-level comparison as it's a combination between streaming and dense



(a) (b) (c) (d) (e)

Figure 8. Visualization of the AttnGate's outputs.

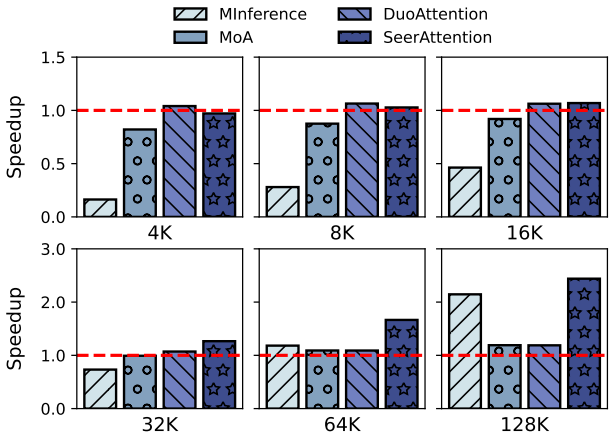


Figure 9. **Comparing Prefilling Time Speedup on RULER Test Setting.** SeerAttention outperforms related works in most long-context data scenarios ($\geq$ 16k). For longer context data, the attention mechanism constitutes a larger proportion of the total runtime, allowing sparse methods to achieve better speedup. Overall, SeerAttention achieves the highest average speedup ($1.41\times$) while maintaining the best average accuracy under this RULER benchmark setting.

head, whose performance is a mixture results of block sparse attention and dense FlashAttention.

Figure 7 shows the sparsity v.s. speedup plots of different methods on 8k, 32k, 128k sequences length, where the speedup baseline is FlashAttention-2. The kernel-level sparsity statistics were collected from PG19 datasets. For MoA, we generated the sparse configurations under their 0.5 overall "KV-sparsity" constraints, which corresponds to an average of 0.35 sparsity in attention. The results demonstrates that the block-sparse attention kernel used in SeerAttention outperforms both MoA and MInference in most cases.

End-to-end Speedup Comparison. To assess the end-to-end speedup of our method, we measured the average prefilling time, or time-to-first-token (TTFT), using the Llama-3.1-8B-Instruct model on the RULER test discussed above. Since attention takes up more runtime with longer contexts, all methods generally achieve better speedup with longer context lengths. It should be noted that SeerAttention uses an identical threshold across all tests in RULER, automatically adjusting to higher sparsity for longer contexts (ranging from approximately 10% sparsity for 4k to around 85% sparsity for 128k). This approach results in an end-to-end prefilling speedup of up to $2.43\times$ on 128k length. On the other hand, MInference experiences a slowdown with data sizes less than 64k due to significant overhead in searching for sparse indices during runtime. It is feasible for SeerAttention to adjust to higher sparsity to achieve greater speedup in shorter contexts as a tradeoff. Nevertheless, SeerAttention delivered the highest average accuracy (87.6) and the greatest average speedup ($1.41\times$) in this RULER benchmark setting.

4.3. Visualization of Learned Attention Maps.

The AttnGate module automatically learns diverse sparse patterns without any prior knowledge or heuristics. Figure 8 shows several example outputs from AttnGate, including (a) "A-shape," or streaming head (b) "Vertical," (c) "Slash" with empty vertical spaces, (d) block sparsity along the diagonal, and (e) random patterns. These patterns not only encompass but also extend beyond those observed in previous works such as MoA and MInference, showcasing the versatillty of our learning based methods.

5. Conclusion and Future Work

This paper presents SeerAttention, a new attention mechanism that learns and leverages the intrinsic sparsity in attention to boost long-context LLMs. SeerAttention learns the attention sparsity from the LLM itself with a lightweight self-distillation approach. Our experiments demonstrate that SeerAttention outperforms previous approaches in terms of long context model accuracy and pre-filling latency. For future work, there are several promising directions to explore for improving and expanding the capabilities of SeerAttention. One key area is enhancing the training methodologies for SeerAttention, such as applying SeerAttention in long-

context continued pre-training with more training tokens to achieve higher sparsity without sacrificing accuracy (preliminary experiments in Appendix A.2). Another important avenue is applying SeerAttention in the decoding stage, especially for long-CoT.

References

Acharya, S., Jia, F., and Ginsburg, B. Star attention: Efficient llm inference over long sequences. *arXiv preprint arXiv:2411.17116*, 2024a.

Acharya, S., Jia, F., and Ginsburg, B. Star attention: Efficient llm inference over long sequences. *arXiv preprint arXiv:2411.17116*, 2024b.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

Brandon, W., Mishra, M., Nrusimha, A., Panda, R., and Kelly, J. R. Reducing transformer key-value cache size with cross-layer attention. *arXiv preprint arXiv:2405.12981*, 2024.

Chen, Z., Sadhukhan, R., Ye, Z., Zhou, Y., Zhang, J., Nolte, N., Tian, Y., Douze, M., Bottou, L., Jia, Z., et al. Magicpig: Lsh sampling for efficient llm generation. *arXiv preprint arXiv:2410.16179*, 2024.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Chuang, Y.-N., Xing, T., Chang, C.-Y., Liu, Z., Chen, X., and Hu, X. Learning to compress prompt in natural language formats. *arXiv preprint arXiv:2402.18700*, 2024.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Computer, T. Redpajama: an open dataset for training large language models, 2023. URL https://github.com/togethercomputer/RedPajama-Data.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. 2023. URL https://arxiv.org/abs/2307.08691.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

Dong, S., Cheng, W., Qin, J., and Wang, W. Qaq: Quality adaptive quantization for llm kv cache. *arXiv preprint arXiv:2403.04643*, 2024.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Fu, T., Huang, H., Ning, X., Zhang, G., Chen, B., Wu, T., Wang, H., Huang, Z., Li, S., Yan, S., et al. Moa: Mixture of sparse attention for automatic large language model compression. *arXiv preprint arXiv:2406.14909*, 2024.

Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Han, I., Jayaram, R., Karbasi, A., Mirrokni, V., Woodruff, D. P., and Zandieh, A. Hyperattention: Long-context attention in near-linear time. *arXiv preprint arXiv:2310.05869*, 2023.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., and Qiu, L. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023.

Jiang, H., Li, Y., Zhang, C., Wu, Q., Luo, X., Ahn, S., Han, Z., Abdi, A. H., Li, D., Lin, C.-Y., et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *arXiv preprint arXiv:2407.02490*, 2024.

Joyce, J. M. Kullback-leibler divergence. In *International encyclopedia of statistical science*, pp. 720–722. Springer, 2011.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pp. 5156–5165. PMLR, 2020.

Kitaev, N., Kaiser, Ł., and Levskaya, A. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Lee, H., Park, G., Lee, Y., Kim, J., Jeong, W., Jeon, M., and Hwang, S. J. Hip attention: Sparse sub-quadratic attention with hierarchical attention pruning. *arXiv preprint arXiv:2406.09827*, 2024.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

Liu, L., Qu, Z., Chen, Z., Ding, Y., and Xie, Y. Transformer acceleration with dynamic sparse attention. *arXiv preprint arXiv:2110.11299*, 2021.

Liu, Z., Wang, J., Dao, T., Zhou, T., Yuan, B., Song, Z., Shrivastava, A., Zhang, C., Tian, Y., Re, C., et al. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*, pp. 22137–22176. PMLR, 2023.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

Mu, J., Li, X., and Goodman, N. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36, 2024.

Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Grella, M., et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

Peng, B., Quesnelle, J., Fan, H., and Shippole, E. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=wHBfxhZu1u.

Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.

Rasley, J., Rajbhandari, S., Ruwase, O., and He, Y. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '20, pp. 3505–3506, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450379984. doi: 10.1145/3394486.3406703. URL https://doi.org/10.1145/3394486.3406703.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.

Tunstall, L., Beeching, E., Lambert, N., Rajani, N., Rasul, K., Belkada, Y., Huang, S., von Werra, L., Fourrier, C., Habib, N., et al. Zephyr: Direct distillation of lm alignment. *arXiv preprint arXiv:2310.16944*, 2023.

Vaswani, A. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Wang, H., Zhang, Z., and Han, S. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 97–110. IEEE, 2021.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., and Han, S. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024.

Yang, L., Zhang, Z., Chen, Z., Li, Z., and Jia, Z. Tidaldecode: Fast and accurate llm decoding with position persistent sparse attention. *arXiv preprint arXiv:2410.05076*, 2024.

Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

Zhang, T., Yi, J., Xu, Z., and Shrivastava, A. Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization. *arXiv preprint arXiv:2405.03917*, 2024a.

Zhang, X., Chang, X., Li, M., Roy-Chowdhury, A., Chen, J., and Oymak, S. Selective attention: Enhancing transformer through principled context control. *arXiv preprint arXiv:2411.12892*, 2024b.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.

Zhu, Q., Duan, J., Chen, C., Liu, S., Li, X., Feng, G., Lv, X., Cao, H., Chuanfu, X., Zhang, X., et al. Near-lossless acceleration of long context llm inference with adaptive structured sparse attention. *arXiv preprint arXiv:2406.15486*, 2024.

A. Appendix

A.1. Training SeerAttention with Customized GPU Kernel

In this appendix, we provide a detailed design and implementation of our efficient kernel, highlighting key modifications to FlashAttention and optimizations for long-context scenarios. We then evaluate the peak memory usage and additional latency overhead of our training kernel during the AttnGate training stage, showing that it incurs only minimal overhead in both memory and latency compared to training with FlashAttention-2.

FlashAttenion with 2D-MaxPooling: A Customized training kernel. In Section 3.2, we discussed the method for obtaining the ground truth attention map used to distill AttnGate. Specifically, we leverage the 2D-MaxPooled attention map from full attention as the ground truth, aligning with the block-sparse attention definition. However, directly computing this attention map is challenging due to the quadratic memory complexity and the fused operation nature of FlashAttention. To overcome this, we developed a customized kernel based on Triton (Tillet et al., 2019) that efficiently extracts the 2D-MaxPooled attention map by modifying the FlashAttention kernel while largely preserving its computation flow. Figure 10 shows the pseudo code and diagram of this customized kernel.

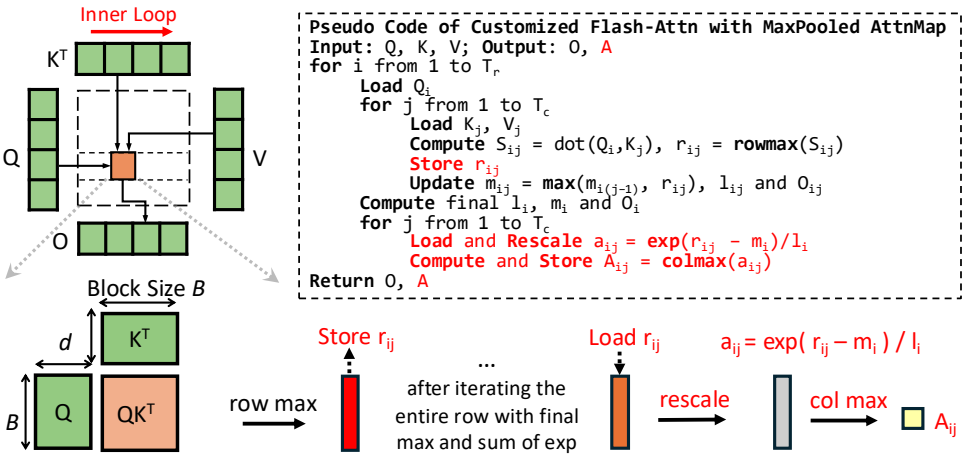


Figure 10. Efficient FlashAttention kernel with pooling of attention map.

Normally, the softmax function ensures numerical stability by subtracting the maximum value before applying the exponential operation. FlashAttention computes the local row max of each block, and gradually updates the global maximum through iteration:

$$\begin{aligned} S_{ij} &= Q_i K_j^T; \\ r_{ij} &= \mathrm{rowmax}(S_{ij}); \\ m_{ij} &= \max(m_{i(j-1)}, r_{ij}). \end{aligned} \tag{5}$$

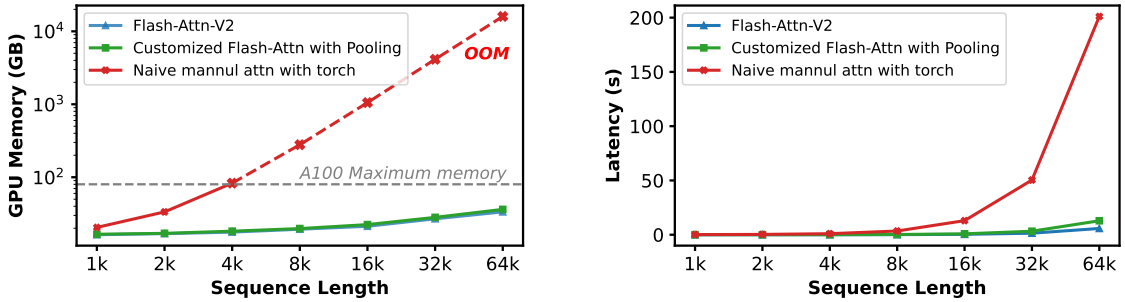
where r_{ij} is typically treated as a temporary result. However, we store it in HBM and rescale it later with the final global max m_i and sum of exp l_i after the iteration:

$$a_{ij} = \exp(r_{ij} - m_i)/l_i \tag{6}$$

This a_{ij} represents the correct row max of the original attention block. With that, 2D-MaxPooling is achieved by applying a column max over a_{ij}. This introduces only minor overhead (storing and rescaling r_{ij}) but significantly improves the efficiency of obtaining the ground truth. The overhead of memory and latnecy analysis is in Figure 11.

Performance of the Training Kernel. We evaluate our customized FlashAttention kernel with 2D-MaxPooled attention map for scalable training of SeerAttention by comparing against with PyTorch naïve manual attention implementation and FlashAttention-2. As shown in Figure 11b, the PyTorch kernel runs out of memory (OOM) when the sequence length exceeds 4k, while our customized kernel costs similar peak memory usage compared to FlashAttention-2. Regarding

latency, since PyTorch encounters OOM for sequences longer than 8K, the attention operations per head into a loop to assess kernel-level latency. Figure 11b shows that the latency overhead introduced by the additional pooling operation is minimal compared to FlashAttention-2, while the PyTorch implementation suffers from a significant slowdown.



(a) Memory (b) Latency

Figure 11. Memory and latency of customized FlashAttention with max-pooling training kernel.

A.2. Preliminary Experiments of Fine-tuning with SeerAttention

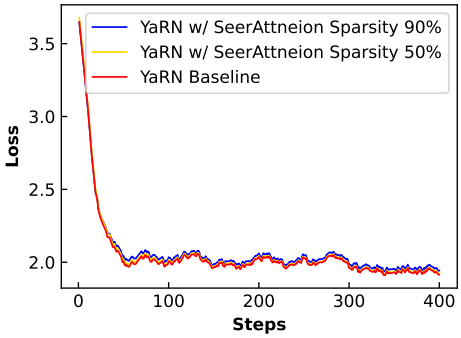


Figure 12. Fine-tuning Loss.

Figure 13. By incorporating SeerAttention with YaRN (Peng et al., 2024) to extend a Llama-3-8B model from 8k to 32k context length, the loss curves for 50% to 90% sparsity are nearly identical to the dense YaRN baseline.

Table 4. Perplexity of YaRN baseline, SeerAttention after YaRN and YaRN fine-tuning with SeerAttention.

	YaRN	Post-training SeerAttention after YaRN					YaRN with SeerAttention				
Sparsity	0.0	0.5	0.6	0.7	0.8	0.9	0.5	0.6	0.7	0.8	0.9
PG19	8.79	9.16	9.30	9.48	9.73	10.18	8.81	8.82	8.85	8.93	9.16
Proof-pile	2.46	2.53	2.57	2.61	2.68	2.85	2.47	2.47	2.48	2.51	2.60

In this preliminary experiment, we demonstrate that SeerAttention can be seamlessly integrated in Long-context extension fine-tuning stages. We follow YaRN (Peng et al., 2024) to extend the context size of a Llama-3-8B model from 8k to 32k. The loss function is the summation of original cross-entropy loss and AttnGate loss. To ensure stable training, the AttnGates are first initialized using the post-training self-distillation before fine-tuning the entire model. We integrate SeerAttention into YaRN and compare the performance against the YaRN dense baseline and the post-training time self-distillation of SeerAttention applied after YaRN. Figure 12 presents the loss curves of the YaRN dense baseline and SeerAttention at 50% and 90% sparsity. The curve at 50% sparsity nearly overlaps with the baseline, while the curve at 90% sparsity shows slightly higher loss. Table 4 displays the test perplexity on the PG19 and ProofPile datasets evaluated at a 32k context length. The YaRN dense baseline achieves perplexity scores of 8.79 and 2.46, respectively. Post-training SeerAttention results in increased perplexity. When applying SeerAttention during the YaRN extension fine-tuning, it maintains near-lossless performance at 50% sparsity (with scores of 8.81 and 2.47), and even at 90% sparsity, the loss remains minimal.